

Hive Monitoring System

Final Design Report

Ryan Bugianesi, Shakthi Sreenivas, Hans Wassermann, Jonathan Wysock, Marwan El Youssoufi

Advisor: Dr. Sasan Haghani

Team Number: SP26-03

Submitted in partial fulfillment of the requirements for senior design project

Electrical and Computer Engineering Department

Rutgers University, Piscataway, NJ 08854

Abstract—Apiculture plays a critical role in global agriculture, with honeybees responsible for pollinating nearly one-third of the food supply. However, recent surveys report unprecedented honeybee colony losses in the United States, exceeding 55% in the 2024–2025 season. These losses disproportionately impact hobbyist and small-scale beekeepers, who often lack the experience and tools needed to detect early signs of colony stress, disease, or collapse. This project proposes a non-invasive, low-cost beehive monitoring system designed to provide real-time insight into colony health through continuous environmental and behavioral sensing. The system integrates temperature, humidity, weight, and acoustic data to capture key indicators of hive condition. Data is collected by a battery-powered microcontroller and transmitted wirelessly to a centralized server using BLE for storage, analysis, and visualization. Machine learning techniques are applied to hive audio and sensor data to infer colony states such as swarming behavior, queen presence, and stress conditions. Data will be presented via a web-based dashboard and insight provided by a context-aware conversational assistant. Using these tools, beekeepers will be able to more effectively manage and understand the needs and health of their colonies.

I. INTRODUCTION

Across the nation, beekeepers do their part in maintaining a healthy population of pollinators; from hobbyist beekeepers who are dedicated to preserving their local ecosystems, to large corporations distributing hundreds of hives to farmers. Beekeeping is a skill derived from experience, where knowing what actions to take, how to respond to difficult-to-spot signs, and managing critical timings are key responsibilities. This makes entering as a hobbyist very challenging. Without proper guidance, new beekeepers may be discouraged by early loss of their colonies.

A. Objective Statement

The goal of this project is to deliver relevant, real-time beehive information to beekeepers through a non-invasive, remote monitoring system. Beekeepers traditionally rely on manual inspections to assess hive health, which can be time-consuming, disruptive to the bees, and insufficient for detecting early signs of stress, swarming, or queenlessness. The system seeks to address these challenges by collecting key environmental data and audio recordings, allowing for continuous and remote

monitoring of hive conditions. The project fills the niche of precision apiculture and data monitoring, providing both new and experienced beekeepers with deeper insight into the state of their colonies. By providing greater insight, the system enhances the accessibility of beekeeping to a broader audience, thereby supporting the preservation of pollinators.

B. Project Objective and Solution

In order to respond to the widespread losses suffered by beekeepers this past year, the team was inspired to create a device to mitigate sudden colony death. As there is still no clear answer as to what caused the spike in loss rate, this project aims to focus on providing additional information to beekeepers for more intelligent management of their colonies.

To address this, the Hive Monitoring System (HMS) is a system capable of gathering, analyzing, and presenting data to a user. Instead of relying solely on raw sensor data, graphs, and charts to communicate the telemetry, the HMS integrates a machine learning driven hive state classifier and a large language model (LLM) conversational agent. These features serve as interfaces through which users can better understand the real-time telemetry gathered by the HMS using natural language.

An array of sensors mounted inside the beehive feeds audio and environmental data to a pretrained transformer-based feature extractor, while a custom-trained classifier head detects important hive signals, such as queen presence. All of this data is presented on an easy-to-use dashboard for beekeepers to track and monitor trends over time.

C. Background and Similar Projects

The HMS's extensive web interface allows for comprehensive and remote monitoring of any beehive where the system is installed. This sets it apart from similar designs which only aim to gather data, or test machine learning models. There seems to be a niche for a remote monitoring solution that beekeepers of all skill levels and number of colonies can utilize. The HMS takes inspiration from the following sources, which all attempt to improve a beekeeper's ability to manage their hives, but fall short on creating a comprehensive remote

monitoring solution. Additionally, with the improvement in artificial intelligence, audio classification tasks are in a position to reduce the number of invasive inspections beekeepers are required to perform on their hives. The HMS attempts to perform all of these functions.

Reference [1] aims to create an extensive remote monitoring system using a Raspberry Pi connected to multiple sensors and a microphone, sending the data over via WiFi to a server, and presenting it on an easily understandable dashboard. A key distinction highlighted by the authors is that beekeepers prefer an easy-to-understand interface over deep data analysis [1]. The HMS helps hobbyists understand what these statistics mean about a hive's health by using ML techniques on the sound samples to aid in colony upkeep and survival.

As far as integrating the hardware to measure hive statistics, reference [2] added a sensor grid layer between the brood box and honey supers using a shim. All the necessary sensors, which are connected to a microcontroller enclosure through the brood box at the bottom of the hive, were mounted inside the shim.

The HMS design takes inspiration from an idea presented in [3], where bait hives were placed near major harbors and ports in Australia, with the intent of giving foreign bees infested with pests a controlled habitat where they could be monitored for health issues without spreading these risks to the mainland; "...it is commonly agreed upon within the honeybee communities that the prevention of an outbreak is better than a cure". These bait colonies were manually inspected on a regular basis, and destroyed when risks were detected. This required a large amount of manpower, specifically from experienced beekeepers who could recognize signs and symptoms of dangerous invasive species, which prompted a desire for automation of this task. The big idea presented here is that, "honeybees change their acoustic behaviour as a result of being exposed to certain stressors". The authors explore means of using acoustic sensing to measure the presence of pests and health-stressed colonies autonomously, which is a direct parallel to the goal of the HMS.

Research into effective methodology of auditory classification of beehive states was done by [4]. The authors identified, compared, and cataloged the effectiveness of multiple edge computing devices (such as Raspberry Pi 5 and Jetson Nano) operating on an array of different convolutional neural networks and classical machine learning models, such as support vector machines and logistic regression, which were trained on publicly available recordings of beehive audio. Instead of relying on edge computing devices, the HMS prefers a centralized approach. Data processing and analysis is performed on a server, rather than at the hive in order to reduce power consumption of the measuring device and utilize more powerful hardware.

Recent developments in artificial intelligence and deep learning have demonstrated that transformer based architectures for audio classification may be more accurate than traditional convolutional neural networks (CNN). [5] showed this by applying a transformer-based neural network architecture

to the task of auditory classification of pollinator species. The researchers observed the transformer architectures outperform the CNN models at this task. This topic reinforces the idea that efforts to build a bioacoustic classifier for beehives could significantly reduce the amount of invasive manual inspections beekeepers need to perform and improve colony survival rates through more informed beekeeping practices, one of the main objectives of the HMS project.

The machine learning system developed for the HMS utilized the data gathered and provided by the Urban Beehive Acoustics and PheNotyping (UrBAN) Dataset [6]. The authors setup a sensing device which recorded audio data that was utilized for training the machine learning inference model. The device used in this setup is similar in concept to the one implemented in the HMS, but lacks a user interface for interacting with and viewing the data. The authors provided detailed notes and labels, however, which were incorporated into the training data used to produce the HMS.

II. METHODS AND RESULTS

Over the past two semesters, the HMS team worked on two major tasks: the design and construction of the beehive monitoring unit, and the development of all accompanying software, including a web application, microcontroller code, and a machine learning inference pipeline.

A. Hardware Design Concepts

The monitoring unit consists of a battery-powered microcontroller connected to an environmental sensor and a microphone. This unit will collect data and transmit it to a secondary microcontroller equipped with a long-range Bluetooth Low Energy (BLE) communication protocol. These two microcontrollers form the foundation of the project, and enable the device to connect to and transmit all the data it gathers for further processing.

The metrics measured include temperature, humidity, atmospheric pressure, weight, and sound. While honeybees regulate their internal hive temperature to around 34°C by clustering for warmth or fanning their wings to cool the hive, extreme outside temperatures can still kill bees. Monitoring temperature in real time helps identify when intervention is necessary.

Similarly, humidity levels play a vital role in brood development and egg hatching, with healthy colonies maintaining a humidity level of approximately 50–60%. Prolonged exposure to drier conditions can hinder colony growth and pose a risk during the winter, as the main source of water when the outside temperature is below freezing is condensation on the inside of the hive walls. Therefore, atmospheric pressure is also an important statistic, as it can give further insight into how much water might be condensing inside a hive.

Weight measurement gives direct insight about honey production and activity levels. It is an important indicator for beekeepers, as supplementary food sources should be added during low activity periods such as autumn and winter months when the bees' food supplies may be low. Significant weight

fluctuations can also be a sign of swarming events, where the colony naturally splits and a large portion of the bees, along with the queen bee, leave the hive in search of a new habitat. The bees bring as many resources with them as possible when leaving, often drastically reducing the survival rate of the remaining colony members. Swarms can often be captured after they leave the hive, so beekeepers could react quickly to such events with real time monitoring insights.

In addition to environmental data, the hive monitoring system collects sound samples from within the hive. These samples will be analyzed using a machine learning model to detect key behavioral patterns such as swarming activity, queenlessness, or colony agitation.

B. Software Design Concepts

As mentioned previously, the usage of artificial intelligence is a key distinguishing factor for the project. With the advent of large language models, there is ample opportunity for humans to interact with machines in more natural ways. This can abstract complex data analysis tasks away for a user of the system. By engaging users through visual representations of data and natural language interaction, the project places a powerful analysis tool directly into the hands of beekeepers at all levels technological competence.

In order to control the hardware described in the previous section, microcontroller code is deployed in order to intelligently manage and modulate all of the devices. The remote sensing microcontroller is programmed to utilize a sleep-wake cycle which maximizes power efficiency. On an interval of 15 minutes, the device wakes itself, initializes sensors, establishes the BLE link, and gathers sensor readings. Once it has completed its tasks, and the transmission is confirmed by the bridge microcontroller, the sensing device returns to its low power sleep state until the next interval passes.

The bridge device is designed to be placed inside a user's home WiFi network. The software listens for the incoming transmission from the sensor device on the BLE long-range channel and relays the information it receives to the cloud server over HTTP API calls.

The centralized server is the main software runner for the project. After acknowledging and receiving the bridge device's API calls, the software sanitizes the incoming data, organizes, and finally stores it on disk. The project's API backend implements receiving the data, managing it on disk, and fetching it once again for use with the website.

The web interface relies on the aforementioned API backend to display the recorded data in any browser via a simple HTML and CSS based web page. Charts are produced plotting the data over time for easy visual recognition of patterns. The web interface also implements a chat window, where users can interact with a LLM hosted on the centralized server. The LLM is context aware and has access to the data recorded by the sensing device and stored on the server via a custom Model Context Protocol (MCP) server built to interact with the API backend.

Finally, the website incorporates a section for displaying machine learning inference results. Each time a sample is recorded to the centralized server, an audio processing task is run to pass the recording through the custom trained machine inference model. The model outputs a single logit representing its confidence that the queen is detected in the audio recording.

The machine learning model was custom trained using Python and the UrBAN Dataset [6]. The model consists of two parts that were trained together. The first part utilizes a pretrained transformer-based model called Audio Spectrogram Transformer (AST) [7]. Audio samples are fed into AST, and the resulting output tensor is passed to a small neural network. The neural network then processes the embedding tensor and returns a logit representing the queen presence confidence. This information is then saved alongside the recorded data for presentation on the web interface.

C. Detailed Design and Results: Hardware

Shown in Figure 1 is a flowchart illustrating how all hardware components of the project work together. The first

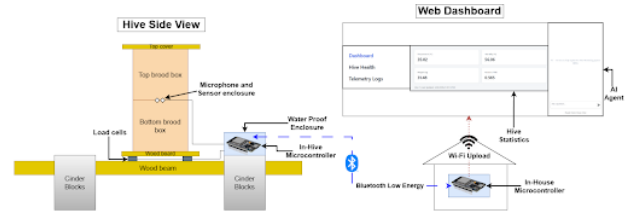


Fig. 1: Project flow diagram

component in the flowchart is the **In-Hive ESP32**, which acts as the central control unit responsible for collecting data from all sensors, processing it, and transmitting it via Bluetooth Low Energy (BLE) to the **In-House ESP32**. The in-house unit serves as an edge device that receives the sensor data and uploads it to the API endpoint via WiFi, where the backend and frontend handle further data processing and display on the web application. This entire procedure is triggered every 15 minutes when the in-hive unit wakes from deep sleep, records the relevant hive data, and transmits it before returning to low-power mode.

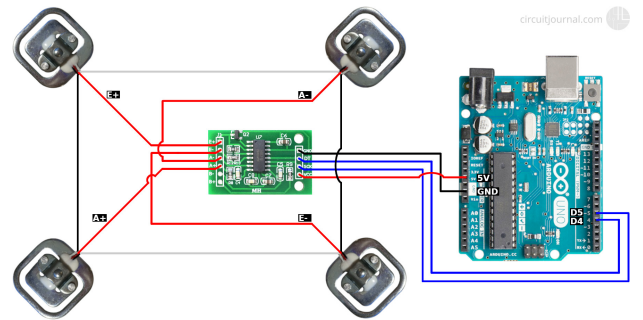


Fig. 2: Four load cell configuration

1) **Weight Sensing:** To measure the total weight of the hive, four 50 kg load cells were mounted beneath the hive structure

and amplified with an HX711 amplifier, as shown in Figure 2. Each load cell operates by slightly deforming under an applied load, causing a change in electrical resistance that produces a small voltage signal proportional to the force applied. The four-cell configuration was chosen to evenly distribute the hive's weight across all four corners, improving measurement accuracy and ensuring no single sensor is overloaded. Weight changes in the hive carry significant meaning, gradual increases over time indicate honey production, while sudden drops may signal swarming or other colony events.

2) *Environmental Sensing*: To monitor the internal conditions of the hive, a BME280 sensor was deployed alongside the microphone inside the hive enclosure. The BME280 measures temperature, pressure, and humidity, providing insight into the environmental conditions experienced by the colony. These readings are particularly valuable because internal hive conditions, such as temperature regulation and humidity levels, are strong indicators of colony health and activity. The BME280 was selected in part due to its compact form factor, which allows it to be placed inside the hive with minimal disruption to the bees.

3) *Audio Recording*: A SPH0645LM4H MEMS microphone was installed inside the hive to capture audio recordings of colony activity. Bee colonies produce distinct sound patterns that vary depending on the state of the hive such as changes in sound intensity, frequency, and overall noise level can indicate normal foraging activity, stress responses, or potential hive issues such as queenlessness. The microphone was chosen for its small size, affordability, and identical specifications as the microphone used by the team that trained the queen status inference model.

4) *Power*: The in-hive system is powered by an 18650 battery housed within the MCU enclosure. The battery supplies sufficient power to the microcontroller and all connected sensors. To extend battery life, the system enters a low-power sleep mode between readings, waking every 15 minutes to collect and transmit a 10–20 second data sample before returning to sleep.

5) *Enclosure and Assembly*: All relevant components were integrated into a custom enclosure designed to make the solution self-contained and robust. Sensor connections were soldered onto a perfboard with header connectors to allow the MCU to be easily disconnected and reconnected for maintenance, as shown in Figure 3. A custom enclosure was designed in CAD and 3D printed to house the in-hive MCU, battery, and battery management module, as shown in Figure 4. The enclosure includes cable routing slits for sensor wires to connect to the perfboard, and the perfboard and battery holder were secured using heat-set inserts. Further waterproofing of the enclosure is planned as a future improvement to better protect the electronics from the humid hive environment.

D. Detailed Design and Results: Software

The microcontroller code was fully implemented as planned and works as described in the Software Design Concepts

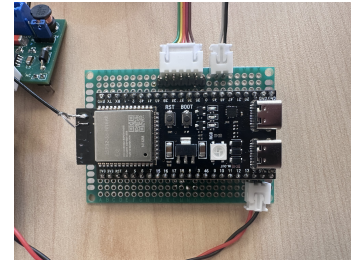
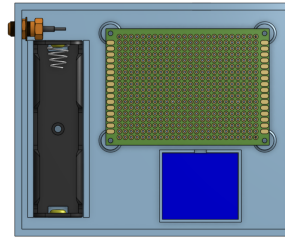
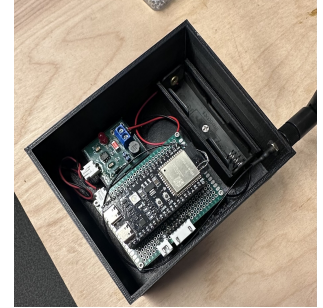


Fig. 3: Perfboard with header connectors



(a) CAD design



(b) 3D printed enclosure

Fig. 4: Enclosure design

section. The code was written using Visual Studio code and dependencies were managed using PlatformIO and ArduinoIDE.

1) *Hive Microcontroller Program*: The program flow for the Hive Microcontroller sensing device is as follows:

The hive microcontroller program is responsible for collecting environmental and acoustic data from within the beehive and transmitting it to the bridge device. The system was designed with a strong emphasis on low power consumption and reliability in remote outdoor environments.

The program operates on a periodic wake-sleep cycle. At fixed intervals of approximately 15 minutes, the microcontroller exits a low-power sleep state and initializes all connected sensors, including the environmental sensor and microphone. Once initialized, the device collects temperature, humidity, pressure, and audio data.

After gathering the sensor readings, the microcontroller establishes a Bluetooth Low Energy (BLE) connection with the bridge device. The collected data is packaged into a structured format and transmitted over the BLE link. The system includes basic confirmation handling to ensure that the data transmission is successfully received.

Once the transmission is complete, the microcontroller shuts down all active components and returns to its low-power sleep state until the next cycle begins. This design significantly reduces overall energy consumption, allowing the device to operate for extended periods on battery power.

2) *In-House Microcontroller Program*: The program flow for the In-House relay device is as follows:

The in-house microcontroller, or bridge device, acts as an intermediary between the hive sensing unit and the centralized

server. Its primary role is to receive data transmitted over BLE and forward it to the backend system over a WiFi connection.

The program continuously listens for incoming BLE transmissions from the hive microcontroller. Upon receiving a data packet, the bridge device parses the information and prepares it for transmission to the server.

The bridge then sends the received data to the FastAPI backend through HTTP API requests. These requests are structured to match the expected input format of the backend, ensuring seamless integration with the data ingestion pipeline.

Unlike the hive microcontroller, the bridge device operates continuously and does not enter a sleep cycle, as it must remain available to receive data at any time. This design ensures reliable data transfer between the sensing unit and the centralized system.

All code for the microcontrollers can be found in the following Github repository, and is freely available for public viewing/usage: [HMS Microcontroller Code](#).

3) *Machine Learning Inference Training and Program*: The machine learning inference project was written in Python and is comprised of a data ingestion part, and a training, validation, and testing loop. The project's high level structure is shown below:

```
configs
  queen_presence_full.toml
  queen_presence_subsampled_05p.toml
data
  embeddings
  models
  processed
  splits
  UrBAN
requirements.txt
run_queen_pipeline.py
scripts
  dataset_extractor.sh
src
  hms_inference
  hms_inference.egg-info
```

The main entrypoint for the project is the `run_queen_pipeline.py` file. This file first loads a configuration file from `configs/`, which contains all the required settings needed to configure the training process of the Audio Spectrogram Transformer and accompanying neural network classifier.

Notable settings configured in this file include the name of the model to be pulled from [huggingface.com](#), specific normalization options pertaining to the Audio Spectrogram Transformer, learning rate, dropout, whether finetuning of Audio Spectrogram Transformer should be attempted, how many layers of Audio Spectrogram Transformer should be “unfrozen” during training, the patience and number of epochs to train for, the threshold to evaluate the confidence logit at during training, and other runtime settings pertaining to the number of Python workers spawned.

After parsing the configuration file, the `run_queen_pipeline.py` script builds the dataset according to the settings in the configuration file. Given the size of the UrBAN dataset, consisting of over 50,000 15-minute audio samples (each segmented into 10-second clips), a 5% subset of the data was utilized to balance training time and model performance. The process of building the dataset involved loading spreadsheets provided by the UrBAN dataset creators containing data labels from beekeeper inspections and associating these notes with the audio files by their closeness in time.

For instance, a typical audio file might be labeled, `31-10-2022_22h45_HIVE-3691.wav`, which corresponds to a sample taken on the October 31, 2022 at 10:45PM. This file would be associated with the closest label from the spreadsheet if inspection data provided by the dataset authors. All chunks created from this 15 minute recording would inherit these labels as well.

To reduce computational load and limit label staleness, only audio recordings within a 24-hour window of the associated label were retained.

After the dataset is built, the results are saved to a `.parquet` file in the `data/processed/<config_filename>` folder, to reduce the runtime of successive trials. The `run_queen_pipeline.py` script then calls another function which cuts out all the data and labels which do not pertain to this main task of queen presence detection. These results are saved to disk again.

Finally, the `run_queen_pipeline.py` script calls the function to begin the training, validation, and testing loop. This function loads the Audio Spectrogram Transformer model, configures it according to the settings, and constructs the small neural network consisting of five layers: A dropout layer, a fully connected layer, a GELU activation layer, a second dropout layer, and a second fully connected layer.

The dataset was partitioned into training, validation, and testing subsets based on Hive ID provided in the UrBAN dataset. This design choice ensures that the model is evaluated on entirely unseen beehive audio, promoting generalization rather than memorization of hive-specific patterns present in the training data. As a result, the model's performance more accurately reflects its ability to generalize to new beehives.

The training function used the balanced accuracy ($BA = \frac{Specificity + Sensitivity}{2}$) as its evaluation metric during the training and validation stage. After each training epoch, the model's performance is evaluated on the validation dataset subset. Then, based on the results of the validation, the program updates the tracked “best model” if the most recent epoch performed better than the previous best. After `patience` number of epochs (a configuration parameter) without improvement or `max_epochs`, the training is considered complete.

The final step before testing the model is to determine the best confidence threshold for the output logit. The best model is loaded once again and tested with the decision threshold swept from 0.05 to 0.95. Since the default threshold was

chosen arbitrarily before training, this final optimization helps ensure the accuracy of the final model. The threshold which maximizes the balanced accuracy is chosen as the best result.

Finally, the model is tested on the test subset of the original dataset. Its performance is evaluated and metrics are saved into files for review.

4) *Machine Learning Results:* Due to time constraints, the HMS only implemented the queen presence detection machine learning inference model. However, the HMS team believes that significant results were achieved, showing the promise of applying the Audio Spectrogram Transformer to audio classification tasks, and showing the efficacy of the idea to evaluate the internal state of a beehive based on audio samples.

The HMS team compared training the model on the full dataset to training the model on a subsampling of 5% of the dataset and found that the subsampled model was actually quite competitive with the fully trained model. Only a few trials were able to be performed due to the complexity and time developing the training program, as well as the extreme time required for a full training run. Using a RTX 3060 12GB graphics card, running the training program on the full dataset took over 6 full days, which blocked further development and testing of the training program. The best result and a snippet of the important settings are presented here:

- Subsample amount: 100%
- Model name: MIT/ast-finetuned-audioset-10-10-0.448
- Dropout: 0.2
- AST do_normalize: false
- max_epochs: 12
- patience: 3
- Finetune: false
- Default Threshold: 0.6
- AST learning rate: 5×10^{-5}
- NN learning rate: 5×10^{-6}
- Best epoch: 4
- Best threshold: 0.95
- Test accuracy: 68.1%
- Test precision: 99.8%
- Test recall: 63.6%
- Test specificity: 99.3%
- Test balanced accuracy: 81.5%
- Test f1 score: 77.7%
- Test confusion matrix:

TP: 20176	TN: 4552
FP: 33	FN: 11528

Key metrics from the training results is the very high precision, or rate of true positives. The model is very good at recognizing cases where the queen is present. When it predicts “true”, it is almost never wrong. This is an important result for the HMS use case; if the precision was lower, the model would be advising beekeepers that the queen is present when, in reality, she is not.

The model also achieves a high specificity of 99.3%, showing strong capability in correctly identifying negative cases, such as potential queen absence. However, this conservative

behavior comes at the cost of recall, which is measured at 63.6%. The relatively high number of false negatives (11,528) indicates that the model may fail to detect some instances of queen presence.

The model achieved an overall accuracy of 68.1%; however, this metric alone does not fully capture performance due to class imbalance within the dataset. A more representative measure is the balanced accuracy of 81.5%, which indicates strong performance across both positive and negative classes.

The HMS team believes that these results show promise that audio analysis of beehives can be used as an identifier for key metrics that would benefit beekeepers. The tradeoff between precision and recall is acceptable for the intended application. The model prioritizes reliability over sensitivity, ensuring that alerts or confirmations provided to users are highly trustworthy. This design choice aligns with the practical needs of beekeepers, where incorrect positive predictions could lead to missed intervention opportunities.

In order to address some of these limitations, the HMS dashboard applies a moving average filter to the results of the queen inference. This improves the readability of the queen presence detection, as multiple consecutive readings are needed in order to change the value of the filter from “Queen Detected” to “Queen not found”.

The repository with the code is publicly available at: [HMS Inference](#).

5) *Web Interface and Supporting Software:* The centralized server hosts not only the machine learning inference part of this project, but also the web interface, large language model for natural language assistant interaction, and API backend. Each of these components builds on the existing data ingestion implemented by the microcontroller sensing devices.

The web interface and supporting backend services serve as the primary interaction layer between the user and the hive monitoring system. The system was designed to provide real-time visibility into hive conditions while remaining simple and accessible for users without technical backgrounds.

The backend was implemented using FastAPI, which handles all incoming telemetry data transmitted from the bridge microcontroller. Upon receiving data, the API validates and structures it into a consistent format before storing it on the server. To support both real-time monitoring and long-term analysis, the system maintains two types of data storage: a continuously updated JSON file representing the current state of the hive, and append-only historical log files that capture sensor readings over time.

The frontend interface is a lightweight web application built using HTML, CSS, and JavaScript. It communicates with the backend through REST API calls to retrieve and display data dynamically. Sensor readings such as temperature, humidity, weight, and pressure are visualized using time-series charts, allowing users to easily identify trends and anomalies.

In addition to data visualization, the interface includes a conversational chatbot powered by a large language model. This chatbot is integrated using the Model Context Protocol (MCP), which enables it to access structured hive data through

API tools. As a result, users can ask questions about hive conditions in natural language and receive responses grounded in real telemetry data.

To evaluate this integration, a series of tests were conducted using common beekeeper-style queries, such as questions about current hive conditions, recent changes in temperature or weight, and queen presence. During these tests, the chatbot leveraged MCP-exposed tools to retrieve real-time and historical telemetry data from the backend, rather than relying solely on prompt-based reasoning. This resulted in responses that were consistently grounded in actual sensor readings, improving both accuracy and relevance. The MCP framework also enabled efficient handling of repeated queries by allowing direct access to structured data, reducing ambiguity and maintaining low response latency. These results demonstrate that MCP significantly enhances the chatbot’s ability to provide reliable, context-aware insights aligned with practical beekeeper needs.

All services, including the API backend, web interface, machine learning inference pipeline, and LLM, are hosted on a centralized server. These components are exposed through a reverse-proxied NGINX configuration, enabling seamless communication between system components and simplifying deployment.

The web interface and backend system were successfully implemented and tested in a controlled environment. The API backend reliably receives and stores telemetry data from the bridge microcontroller, and the frontend interface is able to retrieve and display this data in real time.

Due to time constraints, testing was performed using approximately one day of sensor data. Additionally, the sensing unit is currently deployed in a hive without bees, meaning the collected data represents baseline environmental conditions rather than active colony behavior. Despite this limitation, the system demonstrates stable end-to-end functionality, from data acquisition to visualization.

The chatbot interface was also successfully integrated and is capable of responding to user queries using available telemetry data. This demonstrates the feasibility of combining structured sensor data with natural language interaction to improve accessibility for beekeepers.

Overall, the software system meets its design goals of providing real-time monitoring, historical data access, and intuitive user interaction.

The large language model interaction is managed using an API provided by an external open-source application called AnythingLLM. Incorporating the AnythingLLM application API into the web interface allowed the system to build upon existing large language model chat infrastructure, enabling features such as automatic tool calling, simplified integration with MCP servers, and customizable system prompts through a well-defined GUI. The large language model deployed for this project was a quantized version of Qwen/Qwen3-8B-AWQ deployed through a vLLM docker container.

All of the web services were made available through the use of a groupmate’s existing domain: boojlab.com. Services were

reverse-proxied using a minimal NGINX configuration. Authentication was handled using an identity provider software (Authentik).

E. Website Results and Functionality

The website is organized into sections that show the different areas being monitored within the hive.

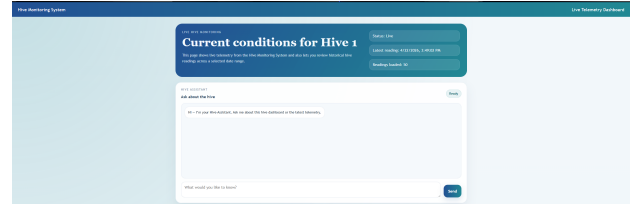


Fig. 5: Chatbot display menu

The following image shows the beginning of the website where the chatbot is located. It also tells which hive it is giving data about if there are multiple hives, the status of the hive, the last reading that was recorded, and how many readings in total were uploaded.

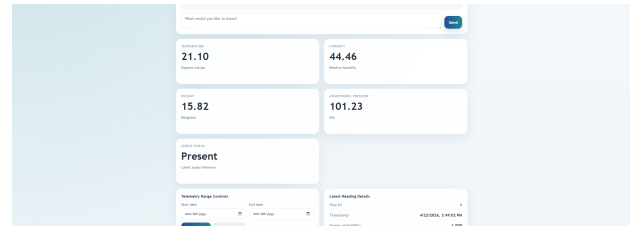


Fig. 6: Telemetry within hive

In the website this is where the data is displayed on the last reading that was done within the hive. This section of the website displays internal hive conditions, including temperature, humidity, weight, and estimated queen presence. A lower panel provides the ability to view data across selected time intervals, along with detailed information from the most recent recorded reading.

Figures 7a, 7b, and 7c show all of the graphs of the different past readings that were recorded and saved within the website. The first three images show the data history of a hive over the course of the day. From this view, overall trends and changes within the hive can be observed. Figure 7d summarizes the readings of each recording to view everything all at once.

F. Software Results Reflection

Overall, this project accomplished the software goals the HMS team set out to achieve. The team presented a compelling interface that beekeepers would enjoy using and find useful when making decisions about managing their beehives. Machine learning services were provided, including a natural language assistant which was context-aware of the sensor readings recorded by the hardware. Graphs were shown depicting trends over time in the sensor readings. And the team displayed

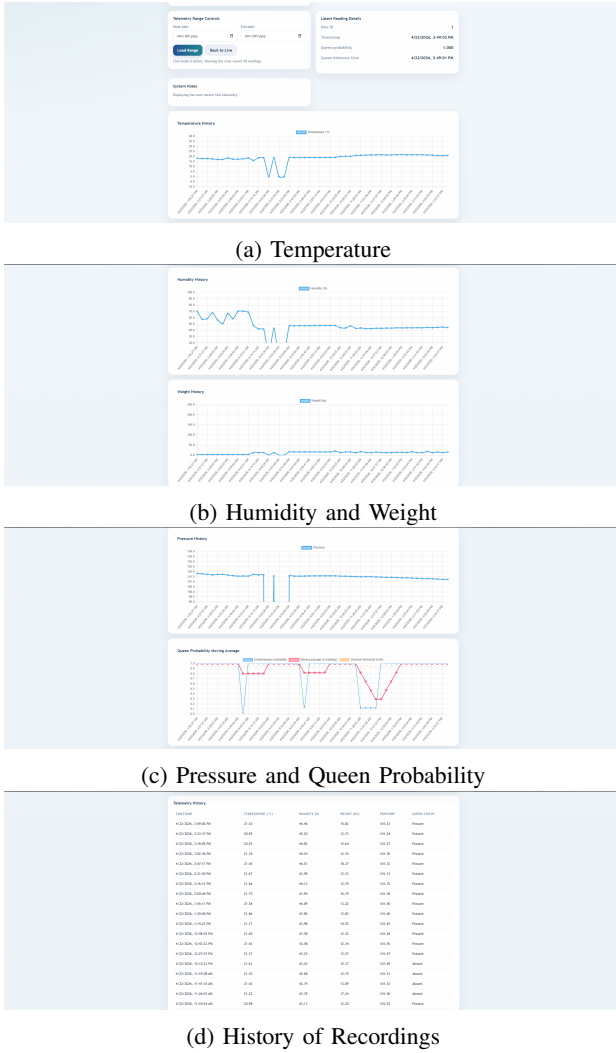


Fig. 7: Graphs and data of previous recordings

machine learning inference results alongside the sensor data to provide additional insight into hive conditions.

Throughout the development of the software components, the majority of effort was dedicated to constructing the machine learning dataset builder, developing the training pipeline, and evaluating different configuration settings. Early in development, a significant challenge was encountered in obtaining meaningful predictions from the trained model, largely due to limited experience with Python, PyTorch, and the Audio Spectrogram Transformer. This issue required extensive iteration, and debugging was further complicated by the long training times of the model.

The root cause was ultimately identified as an optional configuration parameter within the Audio Spectrogram Transformer that applied incorrect normalization to the input data. Once this setting was removed, the model began to show substantial improvement; however, overall progress remained constrained by limited time.

The software components could be further improved

through continued refinement of the machine learning pipeline, additional experimentation with hyperparameters, and the incorporation of larger and more diverse training datasets. While the current results demonstrate promising performance, there remains significant opportunity to enhance accuracy and expand inference capabilities beyond queen presence detection.

III. SYSTEM CHALLENGES AND LIMITATIONS

The development of the Hive Monitoring System presented several technical and practical challenges across both hardware and software components.

One of the primary challenges was structuring and managing incoming telemetry data in a way that supports real-time visualization, historical analysis, and integration with machine learning and large language model components. Ensuring consistency in data formatting across the microcontrollers, backend API, and frontend interface required careful coordination and iterative refinement.

Another major challenge was the development of the machine learning inference model. While the Audio Spectrogram Transformer demonstrated promising results, the model exhibited limitations in accurately generalizing to all cases, particularly for queen presence detection. Additionally, the large size of the dataset and long training times significantly constrained experimentation and model tuning.

System integration also posed challenges, as the project required seamless communication between multiple components, including BLE communication between microcontrollers, HTTP-based data transmission, backend processing, and frontend visualization. Debugging issues across these layers required careful testing and coordination between hardware and software subsystems.

Time constraints were another limiting factor. Due to the scope of the project, certain features—such as expanded machine learning classification tasks, larger-scale data collection, and more advanced user interface enhancements—could not be fully implemented or tested.

Finally, the current dataset used for testing is limited. The sensing device has only collected approximately one day of data, and it is currently deployed in a hive without bees. As a result, the system has not yet been validated under real-world colony conditions, which limits the ability to fully evaluate system performance.

Despite these challenges, the system successfully demonstrates the feasibility of a non-invasive, AI-assisted beehive monitoring platform and provides a strong foundation for future development.

IV. COST AND SUSTAINABILITY ANALYSIS

A. Economic Impact

The prototype for a single in-hive monitoring unit was assembled at a total component cost of approximately \$46.64, broken down as follows:

This cost can be reduced significantly in production through bulk component purchasing, PCB consolidation, and the elimination of development board overhead. At scale, the unit cost

TABLE I: Prototype component cost breakdown

Component	Cost (USD)
2× ESP32-S3 microcontrollers	\$11.90
4× 50 kg load cells	\$8.49
MEMS microphone (SPH0645)	\$11.75
BME280 sensor	\$4.00
3D printed enclosure	\$5.00
18650 Li-ion battery	\$4.00
Battery holder	\$1.50
Total	\$46.64

is expected to decrease substantially. ESP32 chips purchased in quantities of 1,000 units typically cost under \$3 per unit compared to \$6 per development board at retail. Similarly, bulk pricing on sensors, load cells, and enclosures (either through injection molding replacing 3D printing, or contract manufacturing) could reduce the total per-unit cost to an estimated \$20–\$25 at 500–1,000 units. For a commercial beekeeper operating 500 hives, total deployment cost at mass production pricing would be in the range of \$10,000–\$12,500, compared to \$23,000–\$200,000 using existing commercial solutions such as BroodMinder, whose sensors range from \$47.99 to over \$400 per unit depending on capability.

The primary cost saving delivered by this product is in labor. Commercial beekeepers managing hundreds of hives currently rely on manual inspection to assess colony health, a process that is time-consuming and requires experienced personnel. Continuous automated monitoring reduces the frequency of necessary inspections by flagging anomalies in weight, temperature, humidity, and sound, allowing beekeepers to prioritize their time and intervene only when data indicates a problem. Additionally, early detection of hive issues, such as swarming or disease, can prevent colony losses that would otherwise represent significant financial damage.

B. Environmental Impact

This product does not directly emit greenhouse gases or consume significant energy during operation. The in-hive ESP32 spends the majority of its time in deep sleep, waking every 15 minutes for a 10–20 second data collection cycle. This duty cycle results in extremely low average power consumption, and the planned addition of a small solar panel would make the system fully energy self-sufficient, eliminating battery replacement entirely.

The most significant positive environmental impact of this product is indirect: by improving the health monitoring and survival rates of managed honey bee colonies, it supports pollination ecosystems that are critical to global food production. A low-cost, accessible monitoring system that helps beekeepers detect and respond to colony stress early contributes meaningfully to bee population health.

C. Social Impact

This product directly improves the quality of life for both hobbyist and commercial beekeepers by providing continuous, automated insight into hive health without requiring physical inspection. For hobbyist beekeepers, this lowers the barrier to

entry and reduces the expertise required to maintain healthy colonies. For commercial operations managing hundreds of hives, it enables a level of monitoring that would otherwise be logistically impossible, allowing data-driven decisions about hive management at scale.

The system automates the monitoring component of hive management, which is currently performed manually through physical inspection. While this reduces the labor required for routine checks, it does not eliminate the need for skilled beekeepers; it shifts their role from routine inspection toward informed intervention. The product is therefore more likely to enhance the productivity of existing beekeepers than to displace employment. It may also create new opportunities in data analysis, software development, and agricultural technology services.

V. CONCLUSION

The Hive Monitoring System successfully demonstrates a non-invasive, intelligent approach to monitoring beehive health through the integration of hardware sensing, backend infrastructure, machine learning, and natural language interaction. By combining environmental and acoustic data collection with real-time visualization and AI-assisted interpretation, the system provides users with meaningful insights into hive conditions without requiring frequent manual inspections.

The implementation of a web-based dashboard and context-aware conversational assistant enhances accessibility, allowing both experienced and novice beekeepers to better understand and respond to changes within their hives. Additionally, the system architecture supports scalability, enabling deployment across multiple hives with minimal modifications.

While the current system is limited by a small dataset and lack of real-world testing with active colonies, the results demonstrate strong potential for future development. Improvements such as expanded data collection, enhanced machine learning models, and refined user interface features would further strengthen the system’s capabilities.

Overall, this project establishes a strong foundation for precision apiculture systems and highlights the potential of combining embedded systems with artificial intelligence to support sustainable beekeeping practices.

PROJECT REPOSITORIES

- [Microcontroller Code](#)
- [Machine Learning Inference](#)
- [Web Interface](#)
- [MCP Server](#)

REFERENCES

- [1] S. Fiedler, A. Zacepins, A. Kviesis, V. Komasilovs, K. Wakjira, M. Nawawi, O. Hensel, and D. Purnomo, “Implementation of the precision beekeeping system for bee colony monitoring in indonesia and ethiopia,” in *2020 21st International Carpathian Control Conference (ICCC)*, 2020, pp. 1–6.
- [2] J. Navarro, F. Lima, M. Porto, and L. Steinfeld, “Beesmart: a real-time remote monitoring and control system for beekeeping,” in *2022 Symposium on Internet of Things (SIoT)*, 2022, pp. 1–4.

- [3] A. Qandour, I. Ahmad, D. Habibi, and M. Leppard, "Remote beehive monitoring using acoustic signals," *Acoustics Australia / Australian Acoustical Society*, vol. 42, pp. 204–209, 12 2014.
- [4] A. Robles-Guerrero, S. Gómez-Jiménez, T. Saucedo-Anaya, D. López-Betancur, D. Navarro-Solís, and C. Guerrero-Méndez, "Convolutional neural networks for real time classification of beehive acoustic patterns on constrained devices," *Sensors*, vol. 24, no. 19, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/19/6384>
- [5] A. I. S. Ferreira, N. F. F. da Silva, F. N. Mesquita, T. C. Rosa, S. L. Buchmann, and J. N. Mesquita-Neto, "Transformer models improve the acoustic recognition of buzz-pollinating bee species," *Ecological Informatics*, vol. 86, p. 103010, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574954125000196>
- [6] M. Abdollahi, Y. Zhu, H. R. Guimarães, N. Coallier, S. Maucourt, P. Giovenazzo, and T. H. Falk, "Urban: Urban beehive acoustics and phenotyping dataset," 2024. [Online]. Available: <https://arxiv.org/abs/2406.03657>
- [7] Y. Gong, Y.-A. Chung, and J. R. Glass, "Ast: Audio spectrogram transformer," *ArXiv*, vol. abs/2104.01778, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233024831>